

MoE-Infer: A Rust/Metal Engine for Streaming-Expert MoE Inference on Apple Silicon

Claude (Anthropic)* Khanh Nguyen

May 2026

Abstract

We present MoE-Infer, an inference engine for Mixture-of-Experts language models that streams expert weights from SSD on demand and runs all compute through hand-tuned Metal shaders. Starting from the `flash-moe` reference [1] (a C engine that demonstrated 35B-parameter MoE inference on a laptop), we (i) identified and fixed two correctness bugs in the baseline (a norm-shift mismatch between zero-centered and gated RMSNorm, and an MTP embedding indexing bug), (ii) rewrote the engine in Rust with stricter type safety and a “deferred command buffer” design that interleaves adjacent layers into $N+1$ Metal command buffers, and (iii) added a *batched prefill path* that processes N prompt tokens per layer instead of per token, collapsing per-token MoE op2 commits into one commit per layer and loading each unique expert once per layer instead of once per token. On Qwen3.6-35B-A3B-bq4 we measure $2.7\times$ peak prefill speedup vs the token-serial path on Apple M4, and sequential-generation throughput improves from 8 tok/s (`flash-moe`) to 10 tok/s (MoE-Infer) on M1 Pro for the same model. We also report several systematically negative results: low-rank expert factorization fails because Qwen3.6 experts are rank-saturated past layer 0, and rotated quantization (QuIP#-style) does not help against the existing group-wise BF16-scale INT4 scheme.

1 Introduction

Mixture-of-Experts models route each token through only a subset of “expert” subnetworks, activating $K \ll E$ of E total experts per layer. For Qwen3.6-35B-A3B this is $K=8$ of $E=256$ experts per layer. Inference cost scales with active parameters (3B), but storage scales with total parameters (35B). Both `flash-moe` [1] and this work exploit that gap by keeping only currently-active expert weights in GPU memory and streaming them from SSD on demand.

Qwen3.6-35B-A3B’s architecture [2] is hybrid: of 40 transformer blocks, 10 use full multi-head attention with KV cache (every fourth layer) and 30 use Gated DeltaNet linear attention (fixed-size recurrent state). All 40 blocks have a MoE FFN with 256 routed experts plus one shared expert.

This report covers MoE-Infer’s three contributions: correctness fixes relative to the `flash-moe` baseline (§3), sequential-generation optimizations that yield $\sim 25\%$ throughput gain on M1 Pro (§4), and a new batched-prefill engine that yields up to $2.7\times$ on M4 (§5). We also document negative-result experiments (§6) and telemetry-driven analysis of where time goes in the pipeline (§7).

*Implementation, measurement, and writeup by Claude (Opus 4.7). Project direction, architecture decisions, and prior MoE-Infer baseline by Khanh Nguyen.

2 Background

flash-moe. The reference C engine [1] streams 4-bit expert weights via `pread` into Metal buffers, dispatches GPU compute through three sequential command buffers per layer, and runs attention math on the CPU. It reported 8 tok/s on Qwen3.5-35B-A3B on Apple M1 Pro. Its key insight: SSD read bandwidth ($\sim 5-10$ GB/s) suffices to stream $K \cdot S_{\text{expert}}$ bytes per layer per token when overlapped with GPU compute.

Model. Per token, layer:

- Full-attn or linear-attn block: norms, projections, attention compute, residual.
- MoE block: router gate produces softmax over 256 experts; top- K selected; per-active-expert (gate, up, down) projections in INT4; one shared expert in INT4; weighted combine with residual.

Per-layer expert file: E experts $\times (S_{\text{gate}} + S_{\text{up}} + S_{\text{down}})$ bytes. For Qwen3.6 with `moe_intermediate=512`, `hidden=2048`, INT4 + BF16 scales/biases: $S_{\text{expert}} \approx 1.77$ MB.

3 Correctness Fixes

Two real bugs in the baseline, both found by integrating against HuggingFace transformers as the reference (`verify_nway.py`, `helpers/verify-vs-original.py`).

Norm-shift overgeneralization. Qwen3.6 uses *zero-centered* RMSNorm in most places (the quantization pipeline adds +1 to the norm weight, and the kernel computes $x \cdot (w + 1)$) but *gated* RMSNorm inside the DeltaNet block ($x \cdot w$, no shift). The original suffix-match rule `is_norm_key` ended on `.norm.weight`, which matched the gated norm too. The engine then multiplied gated-norm output by $w+1$ instead of w . Fixed by an explicit suffix list and an explicit exclusion of `.linear_attn.norm.weight`.

MTP embedding index bug. The MTP draft path’s CPU `embed_token_cpu` indexed the scales and biases arrays with the wrong base offset (`s[g]` instead of `s_row[g]`), producing garbage embeddings for any token ID > 0 . Fixed by adding the per-row pointer indirection.

Both fixes were verified against the original BF16 model via cosine similarity on logits: the engine now matches the dequantized HuggingFace reference at cosine > 0.9999 on the stripped 4-layer validation model.

4 Sequential-Generation Engine

We rewrite `flash-moe`’s engine in Rust with several design changes that together yield $\sim 25\%$ throughput improvement on M1 Pro for the same model and hardware.

$N+1$ command buffers for N layers. The forward pass has exactly one hard sync point per layer: the router decision (the CPU needs the gate scores before issuing `pread` for the chosen experts). `flash-moe` commits and waits three times per layer. We collapse this to one wait per layer by encoding “op2 of layer N plus op1 of layer $N+1$ ” into a shared command buffer, so each commit covers two layers’ worth of GPU work and the sync barrier amortizes.

Maximal Metal kernel coverage. The original CPU attention math (Q/K head-norm, RoPE, KV cache writes, SDPA, sigmoid gate) is moved to GPU kernels. The engine now has 35 specialized kernels vs `flash-moe`’s 26.

Block-aware quantization (BQ4). A mixed-precision scheme: attention/router weights stay in BF16, `lm_head` in INT8 (per-channel symmetric), expert weights INT4 with per-group BF16 scale/bias at group size 64. Costs $\sim 5\%$ throughput vs an all-INT4 build but recovers most of the quantization gap; see §6 for the analysis.

Application-level expert cache. A small shared LRU on `(layer, expert_idx)` avoids re-streaming hot experts. Size is configurable (default off); 32-entry cache is the sweet spot on M1 Pro (memory cost ~ 54 MB, recovers $\sim 10\%$ throughput). On M4, GPU compute outruns SSD bandwidth even without the cache, so the cache is dead weight there.

Measured. On M1 Pro (10-core CPU, 14-core GPU), Qwen3.5-35B-A3B-4bit, full 40-layer model, 32-token prompt, 100-token greedy generation:

Engine	tok/s
<code>flash-moe</code> (C reference)	8.0
MoE-Infer all-INT4	10.0
MoE-Infer BQ4	9.5

5 Batched-Prefill Engine

For multi-token prompts (chat prefill, RAG context loading), the token-serial engine above leaves much of the GPU idle. Each step dispatches matvecs against weight matrices that are large relative to the input vector, and the GPU’s wide compute units are barely saturated. *Batched prefill* fixes this by processing N tokens together at the same point in the layer stack, sharing weight reads and collapsing per-token command-buffer commits into one per layer.

5.1 Optimization passes

We landed seven distinct optimizations on the branch `batched-prefill`. Each preserves bit-near-identical logits ($\text{max_diff} \sim 2 \cdot 10^{-5}$, top-1 match on all sampled tokens) compared to the token-serial path.

1. **Batched matvec kernels** (`matvec_bf16_n`, `matvec_int8_n`, `dequant_matvec_4bit_n`): Metal kernels taking $[N, \text{in_dim}]$ input and producing $[N, \text{out_dim}]$ output against a shared weight matrix. The grid is linearized (`tgid_flat = row_tile + n · num_row_tiles`) because Metal disallows mixing scalar and vector grid attributes within a single kernel.
2. **Causal batched SDPA** (`attn_sdpa_causal_n`): prefill-side analogue of the single-token fused SDPA. One threadgroup per (i, h) where $i \in [0, N)$ is a new token and h a query head; online softmax over positions $[0, \text{past_pos} + i]$; causal mask is implicit in the loop bound. Paired with `kv_cache_append_n` which writes N K/V vectors at positions $[\text{past_pos}, \text{past_pos} + N)$ in a single dispatch.
3. **op1_full_batched**: full-attention layer’s pre-MoE work for N tokens in one command buffer. Per-token-position kernels (Q/K head-norm + RoPE) loop within the same encoder with offset-into-batched-buffer addressing; the rest use batched matvecs.
4. **op1_linear_batched**: linear-attention layer’s pre-MoE work for N tokens in one command buffer. DeltaNet’s recurrence ($\text{state}_t = f(\text{state}_{t-1}, x_t)$) is sequential by definition, but Metal serializes per-token dispatches via implicit barriers on the shared `conv_state` and `delta_state` buffers. A tiny `buffer_copy_f32` GPU kernel copies between batched and single-token buffers within the encoder (CPU memcopy is not interleavable with encoding because shared-storage writes are only visible at commit).

5. **Single-commit batched MoE op2**: all N tokens’ op2 dispatches encoded into one command buffer per layer, with `encode_post_expert_at` reading per-token slices of `BatchedFullBuffers` via offset arithmetic. Per-token `combine_params` live in `bufs.combine_params_n[ti · 10..]`.
6. **Unique-expert pool**: the largest single win. Without it, $N · K$ expert pre-reads per layer; with it, only the *unique* experts in the batch (up to E) are pre-read once each, then multiple tokens’ `Routing.expert_data` clones point to the same pool slot. For random routing at $N=128, K=8, E=256$: $\sim 200 - 256$ unique vs 1024 pre-reads. Memory cost: $E · S_{\text{expert}} \approx 450$ MB on Qwen3.6.
7. **GEMM-tiled matvec_bf16_gemm_n**: processes `NCOLS_PER_TG=4` tokens per threadgroup with K -axis tiling (`TILE_K=256`). Weight reads are shared across the 4 tokens in a threadgroup, reducing weight bandwidth $\sim 4\times$ on the BF16 matvecs (attention projections, gate, shared-expert projections).

5.2 Results

Measured on Apple M4, Qwen3.6-35B-A3B-bq4 full model. Each row is the minimum of 5 measured runs after one warmup forward.

N	seq tok/s	bat tok/s	speedup	seq ms	bat ms
8	6.51	11.66	1.79 $\times$	1229	686
16	6.29	12.01	1.91 $\times$	2543	1333
32	6.77	18.45	2.73$\times$	4728	1735
64	6.29	14.92	2.37 $\times$	10170	4289
128	6.89	17.82	2.59 $\times$	18582	7182

Notable: speedup grows with N (the unique-expert-pool savings compound), and absolute throughput climbs from ~ 7 tok/s to 12–18 tok/s. Compared to the same engine without batched op2 (*i.e.*, per-token pool, per-token MoE commits), the unique-expert pool alone accounted for a +30–40% jump at large N (Table 1).

Table 1: Incremental contribution of each optimization (speedup vs token-serial baseline, $N=128$).

Cumulative optimizations	Speedup
batched lm_head only	1.05 $\times$
+ <code>op1_full_batched</code>	1.16 $\times$
+ single-commit op2 (per-token pool)	1.62 $\times$
+ batched op1 for linear-attn	1.72 $\times$
+ GEMM-tiled <code>matvec_bf16_n</code>	1.88 $\times$
+ unique-expert pool	2.59$\times$

6 Negative-Result Experiments

We tried two well-known weight-compression techniques. Both failed on Qwen3.6 in ways that are themselves informative.

Low-rank expert factorization. We hypothesized that expert weight matrices $W \in \mathbb{R}^{m \times n}$ might admit a low-rank approximation $W \approx UV^\top$ with $r \ll \min(m, n)$, reducing both storage and pread bandwidth. We measured singular value decay across all 41 MoE layers (script: `helpers/analyze_expert_lowrank.py`). Findings:

- Layer 0 alone is meaningfully low-rank: rank 282 of 512 captures 99% variance, $\sim 28\%$ storage saving.
- Layers 1–40 are rank-saturated: rank > 485 of 512 needed for 99% variance, and the low-rank decomposition is actually *larger* than the dense INT4 matrix it replaces (+21% storage).
- Verdict: low-rank decomposition fails on 40 of 41 layers. The experts have been trained to use their full rank.

Rotated quantization (QuIP#-style). We applied random orthogonal rotation to expert weight columns and re-quantized at INT2 / INT3 / INT4, measuring per-matrix output cosine similarity against the original. Findings: rotation gives $\leq 0.2\%$ cosine lift, *only* at INT2 where the absolute baseline is already broken (cosine ~ 0.91). At INT3 and INT4 the rotation is a wash or slight loss. The per-group BF16-scale quantization (group size 64) already does the local outlier adaptation that rotation would unlock in coarser schemes. Reverting to INT3 alone drops per-matrix cosine from 0.996 to 0.982, which compounds catastrophically across ~ 960 matrix ops per forward, so INT3 is not viable either.

7 Telemetry Analysis

We instrumented the engine to attribute time across GPU wait, disk I/O, and CPU routing (script: `helpers/bench_io.py`). On M4, Qwen3.6 full model, $N=63$ tokens, 5 runs:

Stage	ms (mean)	share
Engine total	9248	100%
GPU op1 wait	4654	50.3%
Expert routing+I/O	3577	38.7%
pread (disk only)	3571	38.6%
Routing CPU (softmax+topk)	8.5	0.1%

Two observations: (i) CPU routing is negligible, validating the decision to not GPU-side the routing (an earlier experiment that did GPU routing regressed throughput by adding 40 Metal dispatches per token). (ii) GPU and disk are nearly balanced ($\sim 50\%$ each), so *either* bottleneck dominates wall time, not their sum — the $N+1$ command-buffer pipelining hides disk I/O behind GPU compute.

The expert cache (§4) is dead weight on M4 because pread already fits inside the GPU shadow. On M1 Pro the GPU is slower and disk is slower; pread becomes the critical path; the cache helps. Hardware-specific.

8 Conclusion

MoE-Infer demonstrates that careful systems work on the GPU/CPU/SSD boundary yields large practical wins for streaming-expert MoE inference. The headline numbers are 25% sequential throughput improvement over the reference engine (10 vs 8 tok/s, M1 Pro) and up to $2.7\times$ prefill speedup via batching on M4. Both come from the same theme: minimize

the number of CPU/GPU synchronization barriers and maximize the work covered per barrier. The negative-result experiments around weight compression (low-rank, rotated quantization) support a complementary observation: on a well-trained MoE the weights are not casually compressible, and the engineering leverage lives in I/O scheduling and dispatch fusion rather than fewer bits per weight.

The code is at https://github.com/fbundle/moe_infer; the batched-prefill engine is on the `batched-prefill` branch.

References

- [1] D. Han. *flash-moe: MoE inference for laptops*. <https://github.com/danielhanchen/flash-moe>, 2024.
- [2] Qwen Team. *Qwen3 Technical Report*. arXiv:2505.09388, 2025.
- [3] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, C. De Sa. *QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks*. arXiv:2402.04396, 2024.
- [4] A. Zandieh, M. Daliri, M. Hadian, V. Mirrokni. *TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate*. arXiv:2504.19874, 2025.